

Função `_generate_orchestrator`

Código completo (303 linhas):

```
def _generate_orchestrator(
    oses: list[dict],
    urls: list[str],
    default_index: int,
    timeout: int,
    system_id: str,
    metadata_url: str,
) -> str:
    """Generate the multi-boot orchestrator bash script."""
    os_count = len(oses)
    os_types = " ".join('"%s"' % e["osystem"] for e in oses)
    url_list = " ".join('"%s"' % u for u in urls)

    return textwrap.dedent('''\
#!/bin/bash
set -euo pipefail

# — Baked-in metadata —
OS_COUNT=%s
OSYSTEM=(%s)
IMAGE_URLS=(%s)
DEFAULT_OS=%s
TIMEOUT=%s
SYSTEM_ID="%s"
MAAS_METADATA_URL="%s"

export WORKING_DIR=/tmp/curtin
export CONFIG=configs/config-000.cfg
export OUTPUT_FSTAB=/tmp/curtin/fstab
export OUTPUT_INTERFACES=/tmp/curtin/interfaces
export OUTPUT_NETWORK_STATE=/tmp/curtin/network_state
export OUTPUT_NETWORK_CONFIG=/tmp/curtin/network_config
export TARGET_MOUNT_POINT=/target/os0
mkdir -p "$WORKING_DIR"

if [ -b /dev/vda ]; then
    DISK_DEV="vda"
elif [ -b /dev/nvme0n1 ]; then
    DISK_DEV="nvme0n1"
elif [ -b /dev/sda ]; then
    DISK_DEV="sda"
else
    echo "[multiboot] ERROR: No valid target block device located!" >&2
    ls -la /dev/
    exit 1
fi

# — Phase 1: Partition once —
curtin -c configs/config-000.cfg block-meta custom

# — Phase 2: Install each OS —
for ((i=0; i<OS_COUNT; i++)); do
    if [[ "${OSYSTEM[$i]}" == "windows" ]]; then
```

```

PART_NUM=$((i + 2))
if [[ "$DISK_DEV" =~ "nvme" ]]; then
    PART_DEV="${DISK_DEV}p${PART_NUM}"
else
    PART_DEV="${DISK_DEV}${PART_NUM}"
fi
START=$(cat "/sys/block/${DISK_DEV}/${PART_DEV}/start")
SIZE=$(cat "/sys/block/${DISK_DEV}/${PART_DEV}/size")
losetup -o $((START * 512)) --sizelimit $((SIZE * 512)) \\\
    --show "/dev/${DISK_DEV}" /dev/loop0
wget "${IMAGE_URLS[$i]}" --progress=dot:mega -O - \\\
    | zcat | dd bs=4M of=/dev/loop0
losetup -d /dev/loop0
partprobe "/dev/${DISK_DEV}"
udevadm settle
else
    if [[ $i -ne 0 ]]; then
        PART_NUM=$((i + 2))
        if [[ "$DISK_DEV" =~ "nvme" ]]; then
            PART_DEV="${DISK_DEV}p${PART_NUM}"
        else
            PART_DEV="${DISK_DEV}${PART_NUM}"
        fi
        mkdir -p "/target/os${i}"
        mount "/dev/${PART_DEV}" "/target/os${i}"
    fi
    CFG_INDEX=$(printf "%03d" $i)
    export CONFIG="configs/config-${CFG_INDEX}.cfg"
    curtin -c "configs/config-${CFG_INDEX}.cfg" extract \\\
        --target "/target/os${i}" "${IMAGE_URLS[$i]}"
    curtin -c "configs/config-${CFG_INDEX}.cfg" net-meta custom
    curtin -c "configs/config-${CFG_INDEX}.cfg" curthooks \\\
        --target "/target/os${i}"
fi
rm -rf /tmp/curtin/*
done

# — Phase 3: GRUB configuration —
cat << 'EOF' > /tmp/execute_phase3.sh
#!/bin/bash
set -euo pipefail
TARGET_DISK="/dev/vda"
EFI_PART="/dev/vda1"
OS1_PART="/dev/vda3"
OS0_MOUNT="/target/os0"
OS1_MOUNT="/target/os1"
HOST_EFI_UUID=$(blkid -s UUID -o value "$EFI_PART" || echo "")
HOST_OS1_UUID=$(blkid -s UUID -o value "$OS1_PART" || echo "")
mkdir -p "$OS0_MOUNT"

# Detect kernel e initrd
KERNEL_FILE=$(ls -l ${OS1_MOUNT}/boot/vmlinuz-* 2>/dev/null | sort -V | tail -n 1
| sed "s|^${OS1_MOUNT}||" || echo "")
INITRD_FILE=$(ls -l ${OS1_MOUNT}/boot/initrd.img-* 2>/dev/null | sort -V | tail -
n 1 | sed "s|^${OS1_MOUNT}||" || echo "")

```

```

# Gera 40_custom
mkdir -p /tmp/grub_payload
cat << PAYLOAD_EOF > /tmp/grub_payload/40_custom
#!/bin/sh
exec tail -n +3 $0
PAYLOAD_EOF

if [ ! -z "$KERNEL_FILE" ] && [ ! -z "$INITRD_FILE" ]; then
    cat << L_EOF >> /tmp/grub_payload/40_custom
menuentry "OS Secundario" --class os {
    insmod ext2
    insmod part_gpt
    search --no-floppy --fs-uuid --set=root ${HOST_OS1_UUID}
    linux ${KERNEL_FILE} root=UUID=${HOST_OS1_UUID} ro quiet splash
    initrd ${INITRD_FILE}
}
L_EOF
fi

if ! grep -q " /target/os0 " /proc/mounts; then
    mount "${TARGET_DISK}2" "/target/os0"
fi
mount --bind /dev "/target/os0/dev" 2>/dev/null || true
mount --bind /proc "/target/os0/proc" 2>/dev/null || true
mount --bind /sys "/target/os0/sys" 2>/dev/null || true
cp /tmp/grub_payload/40_custom /target/os0/etc/grub.d/40_custom
chmod +x /target/os0/etc/grub.d/40_custom

chroot "/target/os0" bash -s << CHROOT_EOF
    mkdir -p /boot/efi
    mount "$EFI_PART" /boot/efi
    sed -i 's/GRUB_DISABLE_OS_PROBER=false/GRUB_DISABLE_OS_PROBER=true/g' /etc/
default/grub 2>/dev/null || true
    sed -i 's/GRUB_TIMEOUT_STYLE=hidden/GRUB_TIMEOUT_STYLE=menu/g' /etc/default/
grub 2>/dev/null || true
    if type -p update-grub >/dev/null 2>&1; then
        grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-
id=ubuntu --recheck "$TARGET_DISK"
        update-grub 2>&1
        cp /boot/efi/EFI/ubuntu/grubx64.efi /boot/efi/EFI/BOOT/BOOTX64.EFI 2>/
dev/null || true
    fi
    umount /boot/efi || true
CHROOT_EOF

umount -l "/target/os0/sys" 2>/dev/null || true
umount -l "/target/os0/dev" 2>/dev/null || true
umount -l "/target/os0/proc" 2>/dev/null || true
umount -l "/target/os0" 2>/dev/null || true
EOF

chmod +x /tmp/execute_phase3.sh
/tmp/execute_phase3.sh

# — Phase 4: Signal and reboot —
wget --post-data="" -q -O /dev/null \

```

```
        "http://${MAAS_METADATA_URL}/MAAS/metadata/latest/${SYSTEM_ID}/?
op=netboot_off"
        (sleep 60; shutdown -r now) >/dev/null 2>&1 &
        exit 0
    ''' % (os_count, os_types, url_list, default_index, timeout, system_id,
metadata_url))
```